TIG-44 : changer l'ordre des arrêts d'une tournée

Documentation du ticket « Cas 4 – Changer l'ordre des arrêts d'une tournée (CommandeChangerOrdre) » – projet Optimod'Lyon

1. Ce que fait le ticket

Après le calcul, l'utilisateur sélectionne un arrêt dans la liste de la tournée (à droite) et clique sur **Monter** ou **Descendre** : l'arrêt échange sa place avec son voisin. Les heures de passage sont recalculées **tout de suite, sans relancer le voyageur de commerce** (règle du 26/10). Le changement est une commande annulable (Ctrl+Z / Ctrl+Y).

Deux refus, avec un message lisible et **rien de modifié** (ni la tournée, ni l'historique) :

- la **contrainte d'ordre** n'est pas respectée : le point de livraison d'une demande passerait avant son point de collecte ;
- le **retour à l'entrepôt** dépasserait 13 h.

L'entrepôt (départ et retour) ne se déplace pas : les boutons restent grisés dessus. Hors périmètre, comme demandé : glisser-déposer, déplacer un point (TIG-43).

2. Ce qui a été ajouté ou modifié

Fichier	Changement	Pourquoi là
model/Plan.java	méthode publique `calculerTourneeOrdreChange` + privée `estArret`	Règle du nettoyage : le calcul (vérification, échange, heures) vit dans Plan, pas dans une nouvelle classe.
controller/CommandeChangerOrdre.java (nouveau)	commande annulable (ADR 0004)	Toute modification faite par l'utilisateur est une Commande, pour qu'Annuler et Rétablir marchent.
controller/Etat.java	méthode par défaut `changerOrdre` (ignorée)	Design pattern State : nouvelle action = une méthode par défaut dans Etat.
controller/EtatTourneeCalculee.java	redéfinit `changerOrdre` et `actionsPermises`	Seul l'état « tournée affichée » permet de réordonner ; il ajoute CHANGER_ORDRE aux boutons actifs.
controller/MapScreen.java	Action `CHANGER_ORDRE`, `definirActionMonterArret`, `definirActionDescendreArret`, `afficherRefusOrdre`	Le contrôleur ne connaît la vue que par MapScreen.
controller/MainController.java	`changerOrdreTournee(rang, decalage)` + branchement des 2 boutons	Le contrôleur garde le cache des tournées et exécute la commande.
view/MainView.java	boutons Monter / Descendre sous la liste de la tournée, fenêtre de refus	La vue transmet l'action et le rang sélectionné ; elle ne modifie pas le modèle.
Tests	PlanTest (+7), CommandeChangerOrdreTest (2), MainControllerTest (+5)	Tests écrits d'abord.

Fichier	Changement	Pourquoi là
README, classes.puml, etats-controleur.puml, ADR 0004	mode d'emploi, diagrammes, commande	Doc dans la même PR que le code (AGENTS.md).

3. Plan.calculerTourneeOrdreChange

`calculerTourneeOrdreChange(Tournee tournee, List<Demande> demandes, int rang, int decalage)`

- **rang** : place de l'arrêt dans la tournée (0 = départ de l'entrepôt, dernier rang = retour). **decalage** : −1 pour monter, +1 pour descendre.
- **Sortie** : la nouvelle `Tournee` avec ses heures de passage.
- **Erreurs** : `IllegalArgumentException` (arrêt qui ne peut pas bouger, contrainte d'ordre) ; `IllegalStateException` si le retour dépasse 13 h (message de `recalculerTournee`).

Étapes, dans cet ordre (le code est commenté de la même façon) :

- 1. Refuser si le rang est le départ ou le retour (« l'entrepôt ne peut pas être déplacé »), ou s'il n'y a pas de voisin dans ce sens (« déjà le premier » / « déjà le dernier »).
- 2. **Contrainte d'ordre, avant tout calcul** : échanger deux arrêts voisins ne change que leur ordre mutuel. Comme la tournée actuelle respecte déjà la contrainte, l'échange n'est interdit que si ces deux arrêts sont le point de collecte (en premier) et le point de livraison (en second) d'une même demande. Un seul test par demande suffit.
- 3. Construire le nouvel ordre (mêmes adresses et mêmes durées, deux éléments échangés).
- 4. Appeler `recalculerTournee` (brique de TIG-101) : A* seulement entre arrêts consécutifs, puis heures de passage. Le départ garde l'heure de départ de la tournée actuelle.

4. Comment une commande s'insère dans le contrôleur

Une tournée n'est pas dans le modèle : le contrôleur les garde dans un cache indexé par la liste des demandes. `CommandeChangerOrdre` reçoit la tournée d'avant, la tournée d'après (déjà calculées) et une petite fonction qui range une tournée dans ce cache à la place de l'actuelle. Donc :

- `executer()` range la tournée d'après ; `annuler()` range celle d'avant. Aucun calcul.
- Le contrôleur appelle `executer(commande)` : l'historique la retient, puis `retrouverEtatProgramme()` réaffiche la tournée du cache. C'est le même chemin que la suppression.
- Si le calcul est refusé, `changerOrdreTournee` affiche la raison par `afficherRefusOrdre` et **n'appelle pas** `executer` : l'historique n'est pas touché, il n'y a rien à annuler.

5. Exemple vérifié à la main (petit réseau de test)

Deux demandes : A collecte en 3 (60 s) et livre en 2 (120 s) ; B collecte en 2 (30 s) et livre en 3 (45 s). Départ 08:00 à l'entrepôt (nœud 1), ordre de la tournée : 1, 3 (A), 2 (B), 2 (A), 3 (B), 1. Les arrêts d'un même nœud se distinguent par leur durée sur place.

Ordre	Retour entrepôt	Résultat
1, 3 (60), 2 (30), 2 (120), 3 (45), 1 (départ)	08:08:39	—

Ordre	Retour entrepôt	Résultat
Descendre l'arrêt de rang 1 : 3 (A) passe après 2 (B)	08:10:51	accepté : 1, 2 (B) 08:01 → 3 (A) 08:03:06 → 2 (A) 08:04:42 → 3 (B) 08:08:18 → 1. Trajet plus long : 1→2 en 500 m au lieu de 1→3 en 100 m.
Descendre l'arrêt de rang 2 : 2 (B) passe après 2 (A)	08:08:39	accepté : ce n'est pas une paire collecte / livraison d'une même demande, et c'est le même nœud, donc aucun trajet en plus (seules les heures sur place changent de place).
Une seule demande (collecte 3, livraison 2), ordre 1, 3, 2, 1 : descendre l'arrêt de rang 1	—	refusé : « Contrainte d'ordre non respectée : le point de collecte (nœud 3) doit être visité avant son point de livraison (nœud 2). »
Cas limite : départ à 12:51, descendre le rang 1	13:01:51	refusé : « retour à l'entrepôt à 13:01:51, après 13:00 » (sans l'échange : 12:59:39).

6. Interface (MainView)

- Sous la liste de la tournée : deux boutons **Monter** et **Descendre**, comme « Supprimer la demande » sous la liste des demandes.
- Ils sont actifs seulement si l'état du contrôleur le permet (`CHANGER_ORDRE`) **et** qu'un arrêt est sélectionné qui n'est ni le départ ni le retour de l'entrepôt (première et dernière ligne).
- Le clic envoie le rang sélectionné au contrôleur (`etat.changerOrdre(this, rang, -1` ou `+1)`). En cas de refus, une fenêtre d'avertissement « Changement d'ordre impossible » affiche la raison.
- Après un changement, la liste est redessinée par le contrôleur ; la sélection est donc perdue et il faut re-sélectionner l'arrêt pour le déplacer encore.

7. Tests

Test	Ce qu'il vérifie
PlanTest.descendreUnArretEchangeAvecLeSuivant	Même résultat qu'un recalcul sur l'ordre échangé (exemple à la main).
PlanTest.monterApresDescendreRedonneLaTournee	Descendre puis monter le même arrêt redonne exactement la tournée de départ.
PlanTest.echangesSurUneTourneeCalculee	Sur la tournée du TSP (moyen plan, 5 demandes) : chaque échange est refusé s'il sépare une collecte de sa livraison, sinon égal au recalcul et réversible.
PlanTest.collecteApresLivraisonRefusee	Refus dans les deux sens (descendre la collecte, monter la livraison), message « Contrainte d'ordre ».
PlanTest.entrepotNonDeplacable, premierArretNeMontePas...	Départ et retour fixes ; premier arrêt ne monte pas, dernier ne descend pas.
PlanTest.retourApres13hRefuseAuChangementDOrdre	Refus à 13:01:51 alors que la tournée rentre à 12:59:39.
CommandeChangerOrdreTest (2 tests)	Exécuter range la nouvelle tournée, annuler l'ancienne, rejouer la nouvelle.
MainControllerTest (5 tests)	Pas de nouveau calcul complet (compteur) ; Annuler / Rétablir ; Monter après Descendre ; refus affiché sans rien modifier ni rien à annuler ; ignoré sans tournée affichée.

Deux tests existants ont été adaptés : quand la tournée est affichée, l'ensemble des boutons actifs contient en plus `CHANGER_ORDRE` (`calculAfficheLaTournee` et le test de suppression avec tournée affichée).

8. Points d'attention

- **Ce qui n'a pas pu être fait ici** : Maven Central et JavaFX ne sont pas accessibles dans l'environnement de travail. Les 129 tests du modèle, des algorithmes et du contrôleur passent (compilés à la main), mais **MainView** n'a pas pu être compilée, et `mvn clean verify`, Checkstyle, SpotBugs, JaCoCo et le rendu des diagrammes sont à lancer en local avant la PR.
- **Arrêts reconnus par nœud et durée sur place** (comme pour la suppression, TIG-101) : une tournée ne garde pas la demande de chaque arrêt. Dans le cas rare où une collecte et une livraison d'autres demandes ont les mêmes nœuds et durées que les deux arrêts échangés, l'échange peut être refusé à tort (jamais accepté à tort). TIG-95 enrichit `Arret` : on pourra alors comparer les demandes directement.
- **Cache des tournées** : le changement d'ordre remplace la tournée stockée pour les demandes actuelles. Cas limite : supprimer une demande, annuler, réordonner, puis supprimer la même demande reprend la tournée déjà gardée pour les demandes restantes (calculée sur l'ancien ordre). La tournée affichée reste valable mais ne garde pas le réordonnancement. Ce cas disparaît quand les tournées vivront dans le modèle (TIG-95).
- **Adaptation à TIG-95** à prévoir dès que le contrat est publié (tournée par livreur : le rang choisi sera celui de la tournée du livreur sélectionné). Branche créée au-dessus de celle de TIG-101 (pas encore mergée) : la PR de TIG-44 doit être mergée après celle de TIG-101.
- **Glossaire** : aucun terme nouveau (arrêt, tournée, point de collecte, point de livraison, contrainte d'ordre, heure de passage).

9. Vérifier à la main

- Lancer l'application, importer `grandPlan.xml` puis `demandeGrand9.xml`, « Calculer ».
- Sélectionner un arrêt au milieu de la liste de la tournée : **Monter** et **Descendre** deviennent actifs. Sur la première et la dernière ligne (entrepôt) : grisés.
- **Descendre** un arrêt : l'ordre change, les heures sont recalculées sans attente. Ctrl+Z remet l'ordre d'avant, Ctrl+Y le refait.
- Essayer de faire passer un point de livraison avant son point de collecte : fenêtre de refus, rien ne change, « Annuler » reste grisé s'il n'y avait rien à annuler.
- Avec un retour proche de 13 h (cas limite) : refus avec l'heure de retour dans le message.